



Prompt: Production-Ready Engineering Guardrails

If you want this editor to feel like a native, enterprise-grade application (and not a sluggish web wrapper), you must strictly adhere to these architectural rules.

1. The Build Pipeline: Do NOT use Create-React-App

A classic beginner mistake is using `create-react-app` and duct-taping Electron around it. CRA is deprecated, slow, and makes compiling native Node modules (like the terminal) a nightmare.

The Production Standard: Use `electron-vite` or `electron-forge` **with the Vite template**.

- **Why?** Vite provides instant Hot-Module Replacement (HMR) during development. More importantly, it correctly splits the build process into two optimized chunks: the Node backend (`main / preload`) and the React frontend (`renderer`).
- **Native Modules:** Your terminal relies on `node-pty` (a native C++ Node module). Vite/Forge will automatically run `electron-rebuild` to compile this C++ code against Electron's specific V8 engine headers, preventing crashes on different operating systems.

2. Monaco Performance: Escape the "React State Trap"

React is built on the concept of re-rendering when state changes. If you store the user's code inside a React `useState` hook and pass it to Monaco, your app will re-render the entire editor on every single keystroke. It will feel incredibly laggy.

The Production Standard: * **Use Refs, not State:** Treat Monaco as an uncontrolled component. Use `useRef` to hold the editor instance.

- **Buffer Management:** When you need the code (e.g., to send to the AI), do not read from React state. Read directly from Monaco's high-performance internal buffer using `editorRef.current.getModel().getValue()`.
- **Resizing:** Do not unmount/remount the editor on window resize. Use a `ResizeObserver` on the wrapper div and call `editorRef.current.layout()` to recalculate the canvas size natively.

3. Security: The API Key Dilemma

Since your editor is AI-powered, users will likely need to input their OpenAI, Anthropic, or local LLM API keys. **Never store these in `localStorage` or plaintext JSON files.** If your app ever suffers an XSS attack (e.g., from rendering malicious markdown in the AI chat), attackers can steal those keys.

The Production Standard: Use Electron's `safeStorage` **API**.

- `safeStorage` encrypts strings using the operating system's native credential managers (Keychain on macOS, DPAPI on Windows, Secret Service on Linux).
- **How it works:** The user enters the key in the React UI -> Sends via IPC to Main Process -> Main Process calls `safeStorage.encryptString(key)` -> Saves the encrypted buffer to disk.

When the AI needs the key, the Main Process decrypts it in memory and makes the API call. The plaintext key *never* lives on the hard drive.

4. IPC Communication: Avoid Memory Leaks

Old Electron tutorials tell you to use `ipcRenderer.send()` and `ipcMain.on()`. This callback-based architecture leads to memory leaks because developers forget to remove listeners (`ipcMain.removeListener`), causing thousands of listeners to pile up during a long coding session.

The Production Standard: Use the **Invoke/Handle Pattern**.

- Use `ipcRenderer.invoke('channel')` on the frontend and `ipcMain.handle('channel')` on the backend.
- This wraps the IPC bridge in standard JavaScript Promises. The React frontend awaits the response, the backend processes it, returns the data, and the connection is garbage-collected cleanly.

5. Process Lifecycle Management (Zombie Hunting)

Because you are implementing the Language Server Protocol (Path B), your Node backend will be spawning background CLI processes (like `pylsp` for Python or `clangd` for C++).

The Production Standard: * If a user force-quits your Electron app, the OS might not automatically kill those child processes. You will leave "zombie" Language Servers running in the background, permanently eating the user's RAM.

- **The Fix:** You must trap Electron's `app.on('before-quit')` and `window-all-closed` events. Keep an array of all active `child_process.spawn` PIDs, and explicitly run `.kill()` on every single one of them before letting the Electron process die.

6. AI Streaming Optimization

When the AI streams code back to the editor, it arrives in tiny chunks (sometimes just 2-3 characters at a time).

The Production Standard:

- Do not update the Monaco document model on every single token arrival. The DOM cannot keep up.
- **Debounce/Batching:** Accumulate the incoming AI tokens into a buffer and flush them to the Monaco text model every 16ms (roughly matching a 60fps refresh rate using `requestAnimationFrame`). This keeps the scrolling and UI butter-smooth while the AI "types".